MongoDB – NoSQL alapok

Szerver · CRUD · Indexek · Aggregáció

Csákó Balázs

Modern adatbázis rendszerek MSc – 10. gyakorlat

Témák

A MongoDB alapok

- Dokumentum-orientált modell
- Gyűjtemények, BSON, `_id`

B Telepítés

- MongoDB Server + mongosh
- JSON import (`mongoimport`)

C CRUD + Indexek

- Insert / Find / Update / Delete
- Szűrők, rendezés, projekció
- createIndex, explain()

D Aggregáció

- Pipeline: `$match`, `$group`, `$sort`
- `$unwind`, `$lookup`

Mi az a MongoDB?

elmélet

Dokumentum-orientált NoSQL

JSON-szerű dokumentumokban (BSON) tárolja az adatokat. Nincs fix séma – minden dokumentum más mezőket tartalmazhat.

Főbb jellemzők:

- Beágyazott objektumok és tömbök
- Automatikus `_id` (ObjectId)
- Horizontális skálázás (sharding)
- **BSON**: Binary JSON – több típus (Date, ObjectId, Decimal128). Limit: 16 MB/dok.

Fogalmi megfeleltetés

Relációs (Oracle)	**MongoDB**
Adatbázis	Database
Tábla	Collection
Sor	Document
Oszlop	Field
PRIMARY KEY	`_id`
JOIN	`$lookup` / beágyazás

Két komponens kell

MongoDB Server – az adatbázis szerver
mongosh – interaktív shell kliens

1. MongoDB Server telepítése

Letöltés:
`mongodb.com/try/download/community`

- Windows: MSI installer (pipáld be: *Install as Service*)
- macOS: `brew install mongodb-community`
- Linux: `sudo apt install mongodb-org`

2. mongosh telepítése (külön csomag!)

```
// Windows (Chocolatey):
choco install mongodb-shell
// macOS:
brew install mongosh
// Linux (Ubuntu):
sudo apt install mongodb-mongosh
// Vagy npm-mel, telepites nelkul:
npx mongosh
```

3. Csatlakozás és adatbázis létrehozás

```
mongosh                    // lokalis
mongosh "mongodb://host:27017"
use vendeglatas            // db valtas
show dbs                   // listazas
```

4. JSON fájl importálása

```
// Termin lban (NEM mongosh-ban!):
mongoimport --db vendeglatas \
  --collection ettermek \
  --file ettermek.json \
  --jsonArray

mongoimport --db vendeglatas \
  --collection szakacsok \
  --file szakacsok.json \
  --jsonArray
```

A `-jsonArray` flag azt jelzi, hogy a fájl egy JSON tömböt tartalmaz (nem soronkénti JSON-t).

CRUD – fogalmak és operátorok

elmélet

CRUD műveletek

Create	`insertOne` / `insertMany`
Read	`find` / `findOne`
Update	`updateOne` / `updateMany`
Delete	`deleteOne` / `deleteMany`

Minden művelet szintaxisa:

- `db.`*`collection`*`.művelet(szűrő, ...)`
- A szűrő egy JSON objektum
- Ha a gyűjtemény nem létezik, **automatikusan létrejön** az első `insert`-nél

Összehasonlító operátorok:

`$gt` / `$gte`	nagyobb / nagyobb-egyenlő
`$lt` / `$lte`	kisebb / kisebb-egyenlő
`$eq` / `$ne`	egyenlő / nem egyenlő
`$in` / `$nin`	benne van / nincs benne
`$exists`	mező létezik-e
`$regex`	reguláris kifejezés

Logikai operátorok:

`$or`	VAGY
`$and`	ÉS (implicit, ha több feltétel)
`$not`	Negáció

Módosító operátorok:

`$set`	mező érték beállítás
`$inc`	növelés / csökkentés
`$push`	tömb bővítés
`$mul`	szorzás

Beszúrás (ha nem importáltunk):

```
db.ettermek.insertOne({
  nev: "Trofea",
  varos: "Budapest",
  cim: { utca: "Visegradq",
         hazszam: "13" },
  csillag: 4,
  specialitasok: ["magyar"]
});

db.ettermek.insertMany([
  { nev: "Gundel", csillag: 5 },
  { nev: "Kistucsok", csillag: 5 }
]);
```

Az _id-t nem kötelező megadni – automatikusan generálódik.

Lekérdezés – `find`:

```
// Minden dokumentum
db.ettermek.find()

// Szures
db.ettermek.find({ csillag: 5 })

// Beagyazott mezo
db.ettermek.find({
  "cim.varos": "Budapest"
})

// Osszehasonlito operator
db.ettermek.find({
  csillag: { $gte: 4 }
})

// Projekcio (nev + csillag, _id nelkul)
db.ettermek.find(
  {}, { nev: 1, csillag: 1, _id: 0 }
)
```

Update:

```
// $set: mezo modositasa
db.ettermek.updateOne(
  { nev: "Trofea" },
  { $set: { csillag: 5 } }
);
// $push: tomb bovites
db.ettermek.updateOne(
  { nev: "Trofea" },
  { $push: { specialitasok: "steak" }}
);
// updateMany: tobb dokumentum
db.ettermek.updateMany(
  { varos: "Budapest" },
  { $set: { regio: "Kozep" } }
);
```

Delete:

```
db.ettermek.deleteOne({ nev: "X" });
db.ettermek.deleteMany({
  csillag: { $lt: 3 }
});
db.ettermek.drop(); // teljes gyujtemeny
```

Rendezés + limit + számlálás:

```
db.ettermek.find()
  .sort({ csillag: -1 }) // 1=ASC,-1=DESC
  .limit(3)

db.ettermek.countDocuments(
  { varos: "Budapest" }
)
db.ettermek.distinct("varos")
```

Miért kellenek?

- Index nélkül: `COLLSCAN` – minden dokumentum végignézve
- Indexszel: `IXSCAN` – gyors keresés
- Az `_id`-ra mindig van automatikus index

Létrehozás:

```
// Egyszeru index
db.ettermek.createIndex({ nev: 1 })

// Osszetett index
db.ettermek.createIndex(
  { varos: 1, csillag: -1 }
)

// Egyedi (unique) index
db.ettermek.createIndex(
  { nev: 1 }, { unique: true }
)
```

Explain – végrehajtási terv:

```
db.ettermek.find({ nev: "Gundel" })
  .explain("executionStats")
```

Fontos mezők

- `winningPlan.stage`: `COLLSCAN` vs `IXSCAN`
- `totalDocsExamined`
- `executionTimeMillis`

Index kezelés:

```
// Listazas
db.ettermek.getIndexes()

// Torles
db.ettermek.dropIndex("nev_1")
```

Koncepció: Lépésenkénti adatfeldolgozás – minden stage alakítja és továbbadja.

```
db.ettermek.aggregate([
  { $match: {
      csillag: { $gte: 4 }
  }},
  { $group: {
      _id: "$varos",
      db_etterem: { $sum: 1 },
      atlag: { $avg: "$csillag" }
  }},
  { $sort: { atlag: -1 } }
]);
```

$unwind – tömb kibontása:

```
db.szakacsok.aggregate([
  { $unwind: "$vegzettsegek" },
  { $group: { _id: "$vegzettsegek",
              db: { $sum: 1 } }}
]);
```

Stage-ek és SQL analógiák:

Stage	SQL analógia
$match	WHERE
$group	GROUP BY
$sort	ORDER BY
$project	SELECT
$limit	FETCH FIRST N
$unwind	(tömb kibontás)
$lookup	LEFT JOIN

$lookup:

```
{ $lookup: {
    from: "ettermek",
    localField: "etterem_nev",
    foreignField: "nev",
    as: "etterem_adatok"
}}
```

Akk.: $sum, $avg, $min, $max, $first

Összefoglalás

Művelet	Szintaxis
Telepítés (shell)	`choco install mongodb-shell` / `brew install mongosh`
JSON import	`mongoimport -db X -collection Y -file Z.json -jsonArray`
Adatbázis váltás	`use vendeglatas`
Beszúrás	`db.coll.insertOne({...})` / `insertMany([...])`
Keresés	`db.coll.find({szuro}, {projekcio}).sort({m:-1}).limit(N)`
Módosítás	`db.coll.updateOne({szuro}, {$set: {...}})`
Törlés	`db.coll.deleteMany({szuro})`
Index	`db.coll.createIndex({mezo: 1})`
Aggregáció	`db.coll.aggregate([$match, $group, $sort, ...])`

Következik a gyakorlati rész